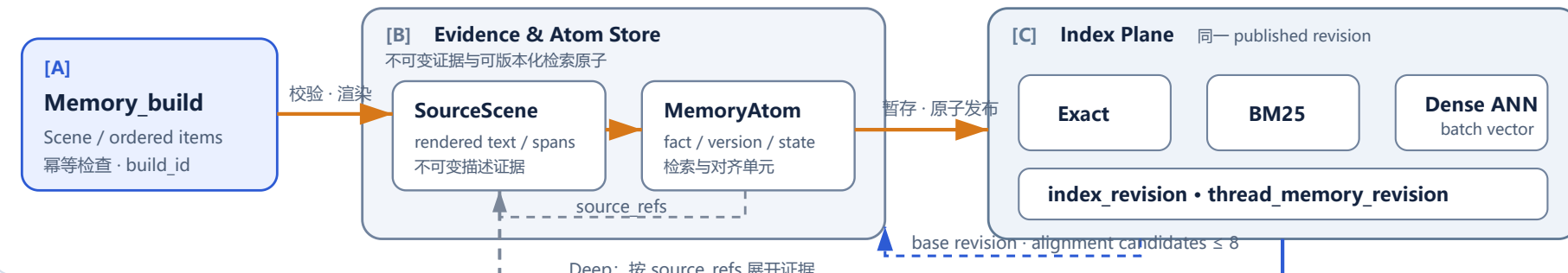


v0.4.1 记忆平台总路线图

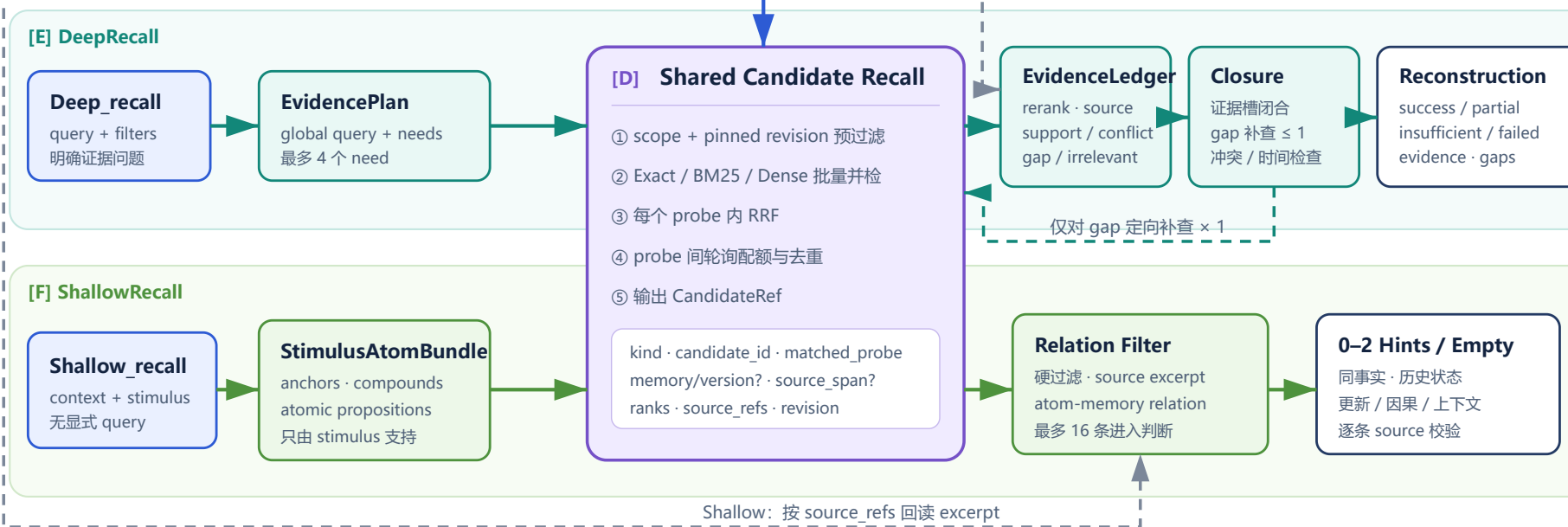
一条构建干线建立证据、原子与索引；Deep / Shallow 固定 revision，共享候选召回核。

构建与发布 · 唯一写路径 SourceScene 保留原始描述，MemoryAtom 负责检索组织，索引与 revision 原子可见



只读召回 · 同一候选核，不同 probe 与最终判定

请求开始固定 index_revision；候选与最终结果均可追溯到原始 item



— 写入 / 原子发布

— 索引读取 / revision

- - - source_refs / 有界回读

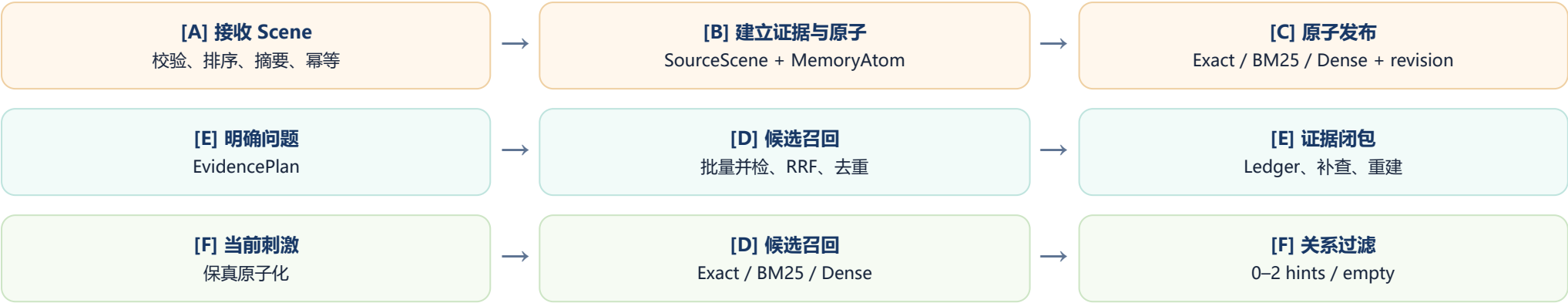
□ Deep 分支

□ Shallow 分支

实线前进 · 虚线回读

先固定主干，再沿 A–F 运行

平台只有一条写路径与两条只读召回路径。每次处理都能在总路线图上落到明确节点，数据对象和 revision 沿箭头传递。



A–F 节点职责

节点	输入	输出
A Memory_build	Scene / items	build_id、构建事务
B Evidence & Atom Store	描述文本	SourceScene、MemoryAtom
C Index Plane	Atom 版本	三类索引、published revision
D Shared Candidate Recall	scope、probes、budget	CandidateRef[]
E DeepRecall	query、filters	reconstruction、evidence、gaps
F ShallowRecall	context、stimulus	0–2 hints 或 empty

全链路固定约束

- `item.content` 是已经形成的可描述文本；不同 `item_type` 使用同一构建流程和同等录入地位。
- `item_type` 只作为渲染标签与来源字段，不参与抽取优先级或排序权重。
- `SourceScene` 是不可变原始描述与证据源，不代表其中每段描述都已被外部验证；`MemoryAtom` 与索引是可重建的派生层。
- 只有 `[A→B→C]` 构建路径可以改变持久状态；`[D]`、`[E]`、`[F]` 全程只读。
- 每个召回请求固定一个 `index_revision`，不读取半发布状态。
- 本方案不包含 `Runtime` 主链、图数据库与实体图处理。



[A] 写路径入口

Memory_build：把一个 Scene 变成可发布事务

一次请求只处理一个 Scene。平台生成 build_id，并以 thread_id + conv_id + scene_id 确定归属与幂等边界。

请求字段

字段	处理
schema_version	选择 0.4.1 解析器；不支持的版本直接失败。
thread_id	硬隔离边界；所有对齐、写入与索引均局限于该 thread。
conv_id	原始对话定位字段。
scene_id	Scene 唯一键与幂等键的一部分。
items[]	按 item_seq 排序的描述文本材料。
memory_proposals	保留接收位置；当前版本不参与任何处理决策。

入口执行序列



构建返回

success + memory_refs
事务已发布；列表是本次涉及的逻辑 memory_id，去重后稳定排序。

success + []
SourceScene 已保存，但没有形成 Atom，或全部抽取结果均为 NOOP。

failed
校验、抽取、对齐、索引或发布未完成；不返回部分成功的 memory refs。

确定性渲染、位置映射与幂等

平台不重写描述语义，只按稳定模板组合各条 `item.content`，同时建立可逆的位置映射。



位置类型

```
item_span
  item_id
  render_start / render_end
  # item 在 SourceScene.rendered_text 中的位置

source_span
  conv_id / scene_id / item_id
  content_start / content_end
  # 事实证据在 item.content 中的位置
```

跨 item 的事实使用多个 `source_span`；不得用一个大范围跨越无关文本。

时间类型

```
source_time = item.occurred_at
event_time  = 原文明示且成功标准化的事件时间
```

- Deep 的 `occurred_from` / `occurred_to` 只过滤 `source_time`。
- 原文没有事件时间时，`event_time` 保持空。
- `item_seq` 只确定当前 Scene 内顺序，不伪造跨 Scene 时间线。

幂等决策表

已有状态	本次 digest	处理	revision
不存在该 Scene	任意有效值	建立新构建事务，继续进入 [B]	发布成功后 +1
同一 Scene 已发布	相同	返回原构建结果或等价 success；不重新抽取	不变
同一 Scene 已发布	不同	返回 failed；不覆盖原始 SourceScene	不变
同一 digest 的事务处理中	相同	合并等待或返回可重试状态；只能有一个发布者	最多发布一次

等同录入： `observation_*`、`thinking`、`tool_use_*` 经过相同的渲染、抽取、对齐和发布步骤；类型不改变事实权重。

[A] Memory_build



[B] Evidence & Atom



[C] Index Plane



[D] Candidate Core



[E] Deep



[F] Shallow

[B] 数据对象

SourceScene 与 MemoryAtom

[B] 同时保存不可变证据与可版本化原子。任何检索结果都必须从 MemoryAtom 回到 SourceScene。

SourceScene: 原始描述证据层

```
thread_id / conv_id / scene_id
schema_version / render_version
items[] / rendered_text / item_spans[]
scene_digest
created_by_build_id
published_revision
```

- 发布后不可修改。
- 即使没有 Atom，也随成功事务保存。
- 保留完整 item 和内容位置，提供 evidence excerpt 回读。
- SourceScene 检索视图只供 Deep 的一次 gap 回退。

MemoryAtom: 检索组织层

```
memory_id / version / state
fact_text
referent / predicate / object_or_value
polarity / modality / temporal_qualifier
anchors[]
event_time_from / event_time_to
source_spans[]
created_by_build_id / published_revision
```

- memory_id 表示逻辑事实谱系；version 标识不可变的事实载荷与 source spans。
- state 为 active / superseded / disputed。
- 结构字段与 fact_text 必须由 source spans 支持。
- 状态变化追加为 transition，并由 revision-scoped state projection 呈现，不原地改写旧版本载荷。

证据约束

自包含

单独读取 fact_text 仍能识别主体、关系与值。

保留限定

否定、数量、持续、时间与模态不得在抽取时丢失。

不超出原文

不创建原文没有表达的意图、目标、计划或未来动作。

可回查

每个 Atom 至少有一个有效 source_span；无来源不得发布。

重建规则：索引、Atom 结构或模型输出损坏时，可从 SourceScene 重建派生层；不能反向用 Atom 覆盖原始 Scene。

从描述文本抽取受证据约束的 MemoryAtom

抽取的目标是形成可独立检索、可对齐、可溯源的最小事实单元。窗口仅控制上下文长度，不改变原文含义。

窗口执行



抽取输出

字段	要求
fact_text	最小且自包含；能由一个或多个 source span 支持。
referent	使用原文名称或唯一可消解指代。
predicate	描述事实、事件或状态关系。
object_or_value	保留对象、数值或状态值。
限定字段	保留 polarity、modality、temporal qualifier。
anchors[]	名称、ID、版本、日期、数字等精确检索锚点。
source_spans[]	精确定位到 item.content 的真实子串。

逐草稿验证

- 1. 检查引用的 item_id 存在且属于当前 Scene。
- 2. 检查 content_start < content_end，且范围不越界。
- 3. 按位置回读真实子串，禁止接受模型回写的替代文本。
- 4. 验证主体、谓词、值、否定、模态和时间均能被引用覆盖。
- 5. 跨 item 原子要求每个桥接信息都有单独 span。
- 6. 删除宽泛标签、纯语气片段以及无法形成事实的草稿。

零原子是合法结果：当前 Scene 仍保存为 SourceScene，并在 [C] 发布新的 revision；memory_refs 可以为空。

同等处理规则

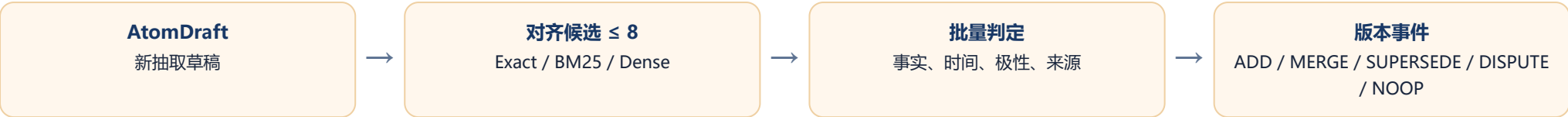
Observation
不自动视为更可靠；按文本与 source span 建立事实。

Thinking
不自动降低录入优先级；只保留文本实际表达的内容。

ToolUse
不自动视为已完成事实；动作、返回与结果依原文分别抽取。

把原子草稿归入逻辑事实谱系

每个草稿先在同一 thread_id 内获取最多 8 个对齐候选，再批量判定写入动作。旧版本从不物理删除。



决策	判定条件	写入行为	最终状态
ADD	没有同一逻辑事实谱系	新建 memory_id, version=1	新版本 active
MERGE_SUPPORT	事实等价，出现新的独立来源	同一 ID 追加合并版本，并追加旧当前版本 superseded / 新版本 active 的状态 transition	projection 中仅新版本 active
SUPERSEDE	原文明示纠正、更新或覆盖	同一 ID 追加新版本，并追加旧版本 superseded / 新版本 active 的状态 transition	projection 中新版本 active
DISPUTE	内容互斥，且没有明确覆盖关系	保留互斥版本与各自来源，并追加 disputed transition	projection 中相关版本 disputed
NOOP	事实与来源已存在，或幂等重放	不生成新版本	不变

时间变化

- “过去是 A，现在是 B”可以形成两个带时间边界的事实，或一次明确状态更新。
- 仅凭发生顺序不同，不执行 SUPERSEDE。
- 自然变化与纠正语句均保留旧版本来源；召回时根据状态和时间解释。

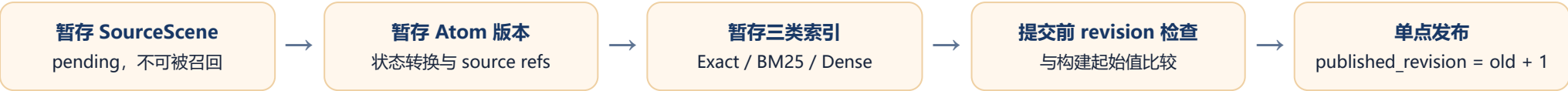
争议处理

- 没有覆盖依据时不自动选择“最新”为真。
- Deep 可读取 disputed/superseded 以重建冲突与变化。
- Shallow 不读取 disputed；只有 active 版本可进入候选。

memory_refs: 返回本事务受影响的逻辑 memory_id；同一 ID 即使产生多个版本也只返回一次。

SourceScene、Atom 与索引按同一 revision 原子发布

抽取和 embedding 可以并行；对齐结果、三类索引与可见 revision 必须一起提交。



并发控制

- 1. 请求开始记录 base_thread_revision。
- 2. 渲染、抽取、span 验证、embedding 可在事务外并行完成。
- 3. 对齐以 base revision 的当前 Atom 作为比较集合。
- 4. 提交前再次读取 thread revision。
- 5. revision 未变化：进入短临界区发布。
- 6. revision 已变化：在新 revision 上重新对齐一次并重建暂存项。
- 7. 再次竞争时进入 per-thread 发布队列，不重复抽取原文。

事务可见性

阶段	Build 可见	Recall 可见
请求接收	build log	否
SourceScene pending	事务内部	否
Atom / index staged	事务内部	否
revision publish	已发布对象	新请求可见
发布失败	failed 日志	旧 revision 继续可读

失败收束

Source 写失败
Atom 与索引均不发布。

Embedding 失败
整个事务失败，不留下只有部分路线的 Atom。

Index stage 失败
清理本 build 的暂存记录；旧 revision 不受影响。

Publish 失败
不推进 revision；重试仍由 scene digest 保证幂等。

[C] 索引平面

Exact、BM25、Dense ANN 与 revision snapshot

三类索引绑定同一个 Atom 版本和 published revision。索引负责快速定位，不取代 SourceScene 证据。

索引	写入字段	命中目标	主要调用
Exact	规范化 anchors、ID、版本、数字、日期、专名	精确同值或别名命中	Shallow anchors; Build 对齐; Deep 明确锚点
BM25	fact_text + anchors, 中文分词与字符 n-gram	复合词、低频实体、字面关系	Deep evidence probes; Shallow compounds
Dense ANN	embed(fact_text)	不同表达的同一原子命题	Deep needs; Shallow rendered atoms

每条索引项的公共元数据

```
thread_id
memory_id / version / state_at_revision
source_scope[]:
  conv_id / scene_id / source_time / source_ref
published_revision
```

thread_id、版本与状态进入检索前过滤; conv/time filter 以 “至少一个 source posting 满足条件” 为准，不能用单一 conv/scene 表示合并 Atom。

Revision Snapshot

```
thread_memory_revision: monotonically increasing
published_index_revision: same visible boundary
request_revision: pinned at recall start
```

- 同一 revision 内三类索引与 Store 一致。
- 旧 revision 可保留到在途读请求结束。
- 新构建发布后，新请求固定到新 revision。

SourceScene 回退索引：可为 SourceScene 建立单独 BM25 / Dense 视图，只允许 [E] 在一次 gap 补查中访问；[F] 不直接搜索原始 Scene corpus。

固定 RecallScope，并只返回可溯源 CandidateRef

Deep 与 Shallow 在请求开始时各自固定 revision；构建中途发布的新记忆不会混入本次候选集。

RecallScope

```
thread_id          # 必填硬隔离
conv_ids?          # 仅 Deep 显式 filters
occurred_from / to?  # 仅过滤 source_time
allowed_states      # 由调用链指定
pinned_index_revision # 请求开始固定
```

Shallow 的 conv_id 只标记调用发生位置，不构成历史搜索边界。

CandidateRef

```
candidate_kind: memory_atom | source_scene
candidate_id
memory_id / version?          # memory_atom
source_document_id / matched_source_span? # source_scene
matched_probe_ids[]
matched_fields[]
route_ranks{}
state?
source_refs[]
index_revision
```

memory_id/version 仅用于 memory_atom；source_scene 使用 source document/span 字段且只出现在 Deep gap 补查。是否足以回答问题或适合成为 hint，由 [E] / [F] 判断。

调用链差异

项目	Deep [E → D]	Shallow [F → D]
检索边界	默认整个 thread；显式 filters.conv_ids 时收窄	整个 thread；当前 conv_id 不收窄
来源时间	显式 occurred filters 贯穿候选、展开和 evidence	无公开时间 filter；只做原子间显式时间兼容检查
Atom 状态	active、superseded、disputed 均可作为重建材料	仅 active；superseded 与 disputed 均不进入 Shallow 候选
当前 Scene	没有协议级排除	当前 scene_id 必须排除
Source corpus	第二轮 gap 补查可访问	不可访问

快照不漂移： 候选、原文回读、rerank、关系判断与最终引用都验证同一 pinned_index_revision；发现引用来自更晚 revision 时丢弃并记录一致性错误。

路线坐标 [C → D] · Pinned Revision

[A] Memory_build



[B] Evidence & Atom



[C] Index Plane



[D] Candidate Core



[E] Deep



[F] Shallow

[D] 共享候选核

唯一共享函数：批量混合召回

Deep 与 Shallow 共享候选检索函数，不共享 probe 生成、证据闭包、关系过滤与输出生成。

```
batch_hybrid_retrieve(  
  scope: RecallScope,  
  probes: list[RetrievalProbe],  
  corpora: set["memory" | "source"],  
  routes: set["exact" | "lexical" | "dense"],  
  budget: RetrievalBudget,  
  exclude_candidate_refs: set[CandidateId],  
  exclude_source_refs: set[SourceRef],  
  deadline_ms: int  
) -> RetrievalBatch
```

1. 预过滤

scope / revision / exclusions



2. 批量准备

Exact 值、lexical query、
dense batch



3. 并行查询

三路索引同启



4. probe 内融合

RRF，保留匹配轨迹



5. probe 间合并

轮询配额、去重、截断

D 负责

- 执行 RecallScope 硬过滤。
- 对全部 probes 做批量 embedding 与批量查询。
- 记录各 probe、各 route 的独立排名。
- RRF、轮询配额与 kind-aware 去重；应用候选和来源排除集合。
- 返回 CandidateRef 与 pinned revision。

D 不负责

- 不生成 EvidencePlan 或 StimulusAtomBundle。
- 不判断候选能否回答 Deep query。
- 不判断候选能否成为 Shallow hint。
- 不生成 reconstruction、evidence、hint。
- 不写回查询结果，不创建记忆。

保留 probe 独立性，再统一合并候选

一个请求内的 probe 不先拼成大查询；每个 probe 独立召回和融合，避免强 probe 吞没弱 probe 的证据槽。

Probe 与路线

probe_kind	Exact	BM25	Dense
Deep global query	有 anchors 时	是	是
Deep evidence need	有 anchors 时	是	是
Shallow anchor	是	否	否
Shallow compound	否	是	否
Shallow atom	否	否	是
Deep source gap	否	是	是

融合伪流程

```
for probe in probes:
    lists = parallel_search(enabled_routes)
    probe_rank = RRF(lists, rank_constant=60)

    pool = round_robin_quota(probe_rank by probe_id)
    pool = dedupe_by_kind(
        memory_atom: memory_id + version + source_ref,
        source_scene: source_document_id + matched_source_span
    )
    pool = truncate(pool, budget.union_limit)
    return candidates + match_trace + revision
```

BM25、cosine 与 RRF 的绝对分数不直接相加；跨 probe 只使用轮询配额和稳定 tie-break。

初始预算

调用链	Probe 上限	每 route / probe	Union	D 之后	轮数
Deep	总 probe ≤ 5 : 1 global + 最多 4 need primary probes	Exact 16 (有 anchors 时)、BM25 32、Dense 32	≤ 96	CrossEncoder 32; source 12-16	1 + gap 补查 ≤ 1
Shallow	anchors ≤ 8 、compounds ≤ 12 、atoms ≤ 4	Exact 16、BM25/Dense 24	≤ 64	硬过滤 / 关系判断 ≤ 16	固定 1

缓存键
thread_id + pinned_revision + probe_hash + filters + model_version

批处理
一个请求内一次 dense batch；Exact、BM25、ANN 并发；rerank 一次 batch。

Deadline
每阶段消费剩余预算；技术超时返回 failed，不把不完整候选伪装成 empty。



[E → D] Deep 第一轮

从明确 query 生成 EvidencePlan

Deep 的检索单位不是一个总结主题，而是一组需要被原始证据填满的 evidence needs。

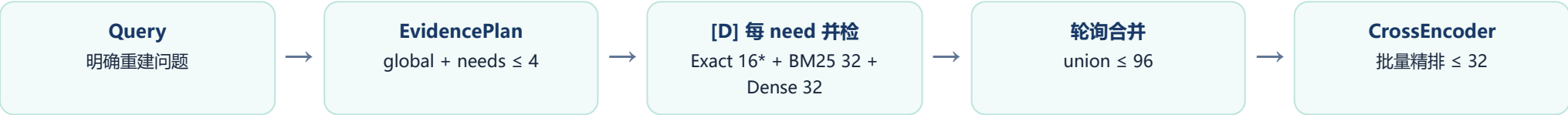
请求校验

字段	规则
schema_version	必须由 0.4.1 解析器支持。
thread_id	硬隔离，不能为空。
query	必须非空且可独立理解。
filters.conv_ids	存在时才限制对话；否则跨 thread 下全部对话。
occurred_from/to	格式有效且 from ≤ to；只过滤原始 source_time。

EvidenceNeed

```
need_id
need_text
required: true | false
primary_probe_text
exact_values[]
dependency_need_ids[]
```

- 最多 4 个 need；每个 need 只生成一个 primary retrieval probe。
- 原始 query 始终作为 global probe。
- 总 retrieval probes ≤ 5；exact values 只是 route 字段，不另计 probe。
- 无依赖 needs 并行执行。
- 推断出的模糊时间只作 probe 文本，不转成硬 filter。



检索与精排：Exact 只在 probe 含 anchors 时启用；每个 evidence_need × candidate.fact_text 独立评分，不能把所有 needs 合并成一个大 query 再精排。

从 CandidateRef 回查 SourceScene，并填充 EvidenceLedger

精排后的 Atom 仍不是最终证据。Deep 只把能回读到真实 item 子串的候选放入 Ledger。



EvidenceLedger

```
need_id
coverage: covered | missing | conflicted
normalized_claim?
event_time?
evidence_entries[:
  evidence_id
  source_ref
  excerpt
  stance
```

Stance 定义

值	含义
supports	原文直接支持该 need 的一个必要事实。
contradicts	原文与待重建事实互斥或明确否定。
context_only	提供人物、时间或桥接背景，但不能独立支持结论。
irrelevant	仅主题相近，不能进入最终 evidence。

回读校验

位置真实

excerpt 必须等于 item.content[start:end]。

范围一致

显式 filters 在回读阶段再次应用。

多来源分列

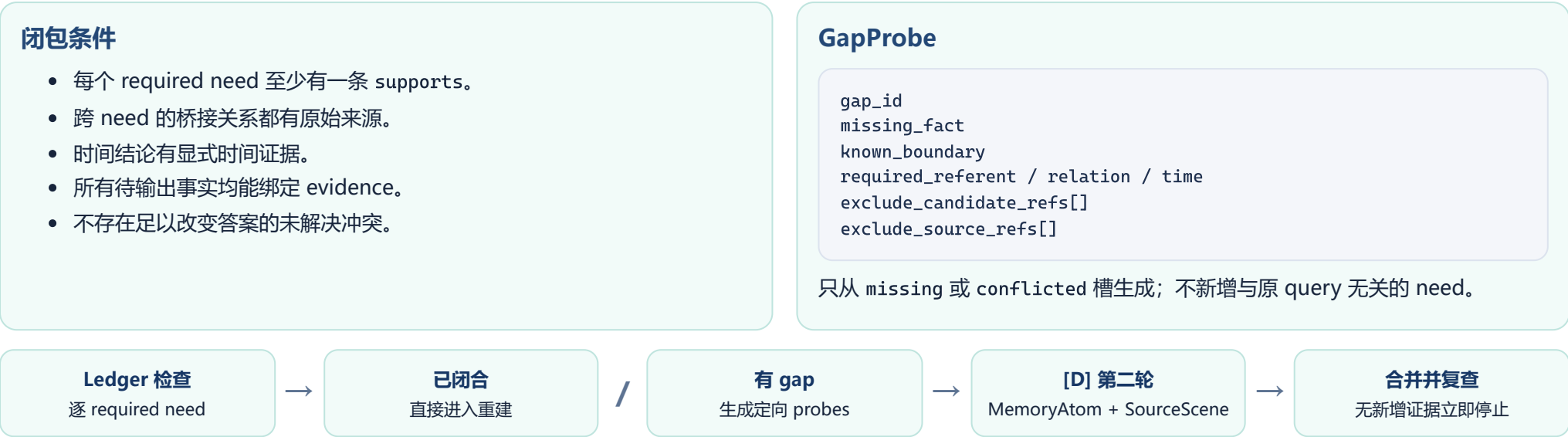
一个 claim 依赖多条 item 时分别生成 evidence。

无来源即淘汰

无法回查的 Atom 不进入 Ledger。

只对未闭合 gap 进行一次定向补查

第一轮之后逐槽判断证据是否闭合。补查目标是填补明确缺口，不是继续发散相关内容。



第二轮硬限制

限制	值	执行规则
补查次数	≤ 1	总检索轮数最多 2；禁止递归 agentic search。
检索语料	corpora={memory, source}	[D] 并行查询两个 corpus 后统一融合；SourceScene 只在该轮开放。
排除集合	第一轮已见候选与来源	减少重复命中，保留 matched trace。
停止条件	无新增有效 evidence	立即停止，不为凑齐答案继续扩展。
写入	禁止	补查和闭包结果不更新 MemoryAtom。

逐主张验引，再确定 success / partial / insufficient

重建器只能消费 EvidenceLedger。最终 evidence 只保留 reconstruction 实际引用的原始材料。



status	reconstruction	evidence	conflicts / gaps
success	非空，所有事实已验引	非空，只含被引用来源	没有影响结论的缺口或冲突
partial	非空，只含已证实部分	非空	至少一个明确 gap 或 material conflict
insufficient	null	[]	记录不能形成可靠重建的缺口或冲突
failed	null	[]	通过 error_code / error_message 描述技术或输入错误

验引通过

- evidence id 存在且 source_ref 可回读。
- excerpt 直接支持主张，不只主题相关。
- 时间、数量、否定与归属均未超出来源。
- 重建文本中的每个事实性句子带引用。

必须剔除

- 模型从多条证据间补出的无来源桥接。
- 把 disputed 内容静默裁决为单一真相。
- 把检索排名或模型置信度当成事实支持。
- 为了避免 insufficient 而保留未经验证的文本。

[A] Memory_build



[B] Evidence & Atom



[C] Index Plane



[D] Candidate Core



[E] Deep



[F] Shallow

[F] Shallow 输入

以当前 stimulus 为唯一激活源

Shallow 不接收 query。最后一条 item 是当前刺激，之前的 items 只用于有限消解与重复判断。

请求边界

```
stimulus = scene_items[-1]
context = scene_items[0:-1]
```

- scene_items 非空，item 格式与 Memory_build 相同。
- 任意 item_type 都可成为 stimulus。
- conv_id 标记当前调用位置，不限制历史检索。
- 当前 scene_id 必须从来源中排除。
- 请求不暴露 query、top_k、来源 filter 或排序阈值。

Context 允许的处理

代词消解 唯一省略主语补全 相对时间标准化 Scene 重复检测

Context 禁止的处理

生成新检索主题 推断意图 推断目标或计划 生成未来动作 拼接全部上下文检索

只有指向唯一时才允许补全；存在多个候选时保留未解析形式。

StimulusAtomBundle 生成

读取 stimulus.content
不拼接全部 context



保留限定
否定、数量、时间、持续、模态



anchors ≤ 8
精确身份和值



compounds ≤ 12
有区分度的组合词



atoms ≤ 4
自包含原子命题

关键约束： anchors、compound terms 与 atoms 都必须由 stimulus 支持；“意图”不进入检索输入。

原子信息分别进入 Exact、BM25 与 Dense

每类关键信息只进入最匹配的检索路；所有 probes 批量提交，三类索引并行执行。



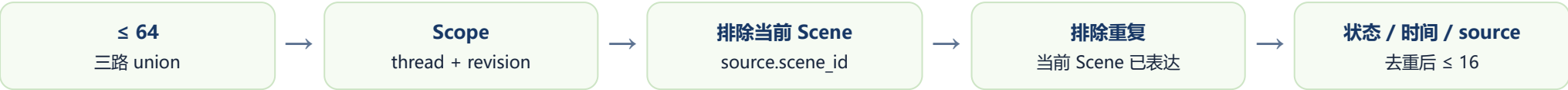
提速执行表

阶段	批量化	并行化	提前停止
Stimulus 处理	一次结构化输出全部 probes	—	没有有效 anchors/compounds/atoms → empty
Embedding	atoms 一次 batch	可跨请求 micro-batch	没有 atoms 则跳过 Dense
检索	每 route 多 probe 批查	Exact、BM25、ANN 同时运行	某类 probe 为空则跳过该 route
过滤	候选一次批读 metadata/source	可与去重流水执行	依赖正常且语义过滤为空 → empty; 读取错误 → failed

先做确定性硬过滤，再判断原子—记忆关系

“有意义” 被定义为刺激原子与历史原子之间的明确关系，而不是对未来响应意图的预测。

硬过滤漏斗



relation	保留语义	示例判断边界
same_fact	同一事实或同一事件的历史来源	不是只共享实体；核心谓词和值一致。
historical_state	同一对象或属性的旧状态	需要可解释的时间边界。
update_or_conflict	明确持续、更新、纠正或冲突	未解决 disputed 不直接生成确定性 hint。
cause_or_result	原文明示的原因或结果	不能从相关事件自行推断因果。
reference_context	补全刺激中的明确指代对象	只有唯一指向时保留。
none	没有可陈述关系	仅话题、实体或宽泛词重合。

保留条件

```
relation != none
AND 候选提供当前 Scene 没有的信息
AND 时间 / 版本关系可以清楚表述
AND hint 能由原始 source 支持
```

直接删除

- 只有共同人物、产品或话题。
- 候选只是当前 stimulus 的同义复述。
- 显式时间与 stimulus 原子不兼容。
- 非 active 候选。

完整性边界：source_refs 缺失、无法回读、字符范围越界或 revision 不一致属于数据完整性/技术错误；若因此无法完成候选验源，本次调用返回 failed，不得降级为 empty。

关系去重、限额与逐条 source 校验

Shallow 只有一轮召回。通过关系判断的候选最多生成两条短 hint；没有合适内容时正常弃权。



Hint 输出约束

- 每条包含 hint_id、text、source_refs[]。
- text 表明信息归属，必要时标记“此前”“当时”“后来”等时间边界。
- 不表达推断的用户意图，不生成行动建议。
- 不裁决未解决冲突，不把 disputed 压缩成单一事实。
- 每条 text 逐字检查核心事实是否由 source excerpt 支持。

状态判定

status	hints	条件
hit	1-2	至少一条完成关系与来源校验。
empty	[]	所有必需阶段均成功，且无有效 atom、无候选、语义过滤为空或 relation 全为 none。
failed	[]	输入校验失败、索引不可用、source 完整性错误、超时或其他技术失败。

单轮运行预算

24

最多 probe 数: 8 anchors + 12 compounds + 4 atoms

64

候选 union 上限

16

进入 source / relation 批处理上限

0-2

最终 hints; 无结果返回 empty

构建示例：同一 Scene 同时产生 ADD、MERGE 与 NOOP

示例用于展示数据如何沿总路线图流动；文本只是说明性样例。

[A] 输入 Scene

```
item-1 observation_user:
"LoCoMo 数据导入后日期偏移一天。"

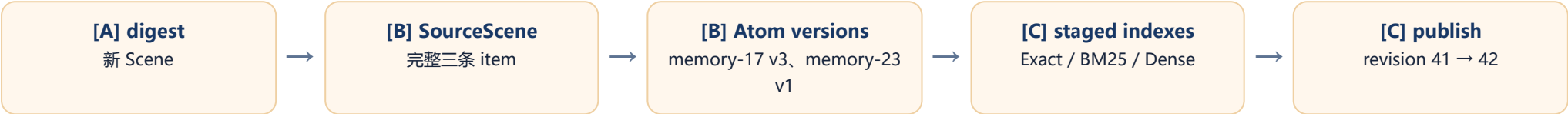
item-2 thinking:
"这个问题此前也出现过。"

item-3 tool_use_parser:
"使用 UTC+8 重新解析后，日期恢复正确。"
```

排序后渲染，记录每个 item_span，计算 scene_digest。

[B] Atom 草稿与对齐

草稿	决策	结果
导入后日期偏移一天	MERGE_SUPPORT	旧 memory-17 新增来源
UTC+8 重解析后日期恢复正确	ADD	新建 memory-23
此前也出现过	NOOP	缺少可独立事实，不发布 Atom



```
{
  "build_id": "build-20260810-042",
  "status": "success",
  "memory_refs": ["memory-00017", "memory-00023"]
}
```

可见性： revision 42 发布前，在途召回继续读取 41；发布后新召回固定到 42，并同时看到 SourceScene、两个 Atom 版本和三类索引项。

Deep 示例：第一轮缺少原因，第二轮完成闭包

Query 被拆成三个 evidence needs; 只为缺失的“原因”槽执行一次 SourceScene 补查。

[E] EvidencePlan

1. N1: 此前做了什么决定?
2. N2: 决定的适用版本是什么?
3. N3: 明确表达的原因是什么?

global probe 保留完整原 query。

[D] 第一轮

BM25 + Dense → union → rerank。

- N1: covered
- N2: covered
- N3: missing

Ledger 中没有可支持原因的 excerpt。

[E → D] GapProbe

missing_fact: 不接入 Runtime 的明确原因

known_boundary: v0.4.1 先建设独立平台

corpora: MemoryAtom + SourceScene

第一轮 Ledger

N1/N2 支持, N3 缺失

一次 gap 补查

排除已见来源

新 evidence

命中原 Scene 说明

闭包复查

三个 required needs 均 covered

逐主张验引

success

```
{
  "recall_id": "recall-20260810-deep-001",
  "status": "success",
  "reconstruction": "v0.4.1 先建设独立记忆平台, 不接入 Runtime 主链; 接入工作留到后续版本。[E1][E2]",
  "evidence": [
    {
      "evidence_id": "E1", "conv_id": "conv-008", "scene_id": "scene-021", "item_id": "item-004",
      "item_type": "observation_user", "occurred_at": "2026-08-08T16:20:00+08:00",
      "excerpt": "现在不要录入到 Runtime 链路, 先搭系统。",
    },
    {
      "evidence_id": "E2", "conv_id": "conv-008", "scene_id": "scene-021", "item_id": "item-005",
      "item_type": "observation_user", "occurred_at": "2026-08-08T16:21:00+08:00",
      "excerpt": "Runtime 接入留到后续版本。"
    }
  ],
  "conflicts": [],
  "gaps": []
}
```

Shallow 示例：三路命中后只保留一条历史线索

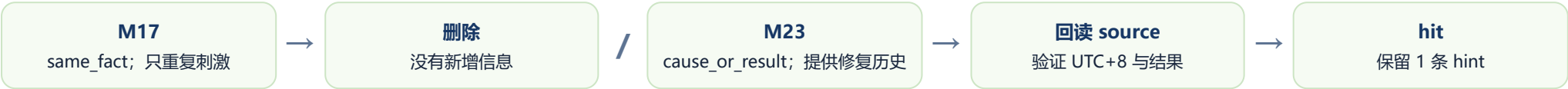
当前刺激只提供关键信息；上下文用于消解“它”与检查当前 Scene 是否已经包含候选事实。

[F] StimulusAtomBundle
stimulus: “LoCoMo 导入后日期还是偏移一天。”

anchors:
["LoCoMo", "一天"]
compound_terms:
["LoCoMo 日期", "导入 日期", "日期 偏移一天"]
atoms:
["LoCoMo 数据导入后, 日期仍偏移一天。"]

[D] 候选结果

候选	命中路线	硬过滤
M17: 此前也偏移一天	Exact + BM25 + Dense	保留
M23: UTC+8 修复过	BM25 + Dense	保留
M31: 另一个数据集的日期问题	BM25	主体不一致, 删除
当前 Scene 自身	Dense	scene_id 相同, 删除



```
{
  "recall_id": "recall-20260810-shallow-001",
  "status": "hit",
  "hints": [{
    "hint_id": "H1",
    "text": "此前同一导入问题曾在改用 UTC+8 重新解析后恢复正常。[S1]",
    "source_refs": [{
      "source_ref_id": "S1", "conv_id": "conv-008", "scene_id": "scene-021",
      "item_id": "item-003", "item_type": "tool_use_parser",
      "occurred_at": "2026-08-08T16:10:00+08:00"
    }]
  }]
}
```

状态分支: 如果 M23 时间不兼容或 relation 只能判为 none, 返回 empty; 如果来源无法回读或 revision 不一致, 返回 failed。

示例轨迹 [F → D → F]

Revision、缓存、日志与状态收束

控制面不新增公开接口，只保证 A–F 在同一版本边界、延迟预算和追踪语义下运行。

运行记录

记录	最小字段
Build log	build_id、scene digest、base/published revision、Atom 决策、耗时、错误
Recall log	recall_id、pinned revision、probe 数、route 命中数、截断数、状态
Evidence audit	claim、evidence ids、excerpt hash、验引结果
Hint audit	stimulus atom、candidate、relation、保留/删除原因、source refs

缓存与失效

```
cache_key =
  thread_id
+ pinned_index_revision
+ probe_hash
+ filters
+ embedding/reranker/model_version
```

- 新 revision 发布后，旧键自然不再被新请求使用。
- Source excerpt 缓存按 source_ref 与 scene_digest 校验。
- 缓存命中不绕过 scope、revision 与 source 校验。

状态收束矩阵

链路	正常有结果	正常无完整结果	技术 / 输入失败	是否写记忆
[A→B→C] Build	success + refs	success + []	failed	仅 success 发布
[E→D→E] Deep	success	partial / insufficient	failed	否
[F→D→F] Shallow	hit	empty	failed	否

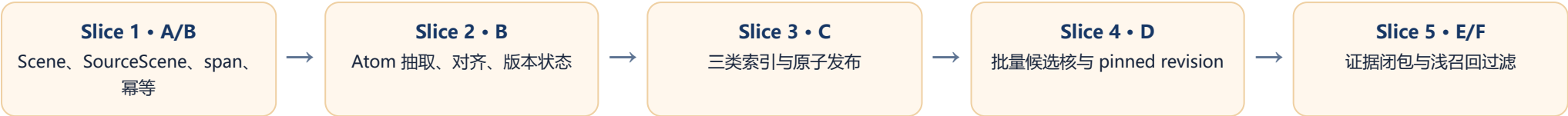
一致性指标
跨 revision 候选数必须为 0；source_ref 回读成功率必须为 100%。

召回指标
分开记录候选召回、过滤淘汰、最终 evidence/hint 支持率。

延迟指标
按 atomization、embedding、三路检索、rerank、source read、验证分段记录 p50/p95。

按路线图分层实现与验收

每一层只依赖前一层已经稳定的产物；Deep 与 Shallow 在 [D] 稳定后并行接入。



节点	最小可验收结果	关键失败测试
[A]	同 Scene 同 digest 幕等；不同 digest 拒绝覆盖	重复 item_seq、非法时间、跨 thread 标识复用
[B]	每个 Atom 可回读真实 source span；版本事件可重放	越界 span、丢失否定、无依据 SUPERSEDE
[C]	Store 与三索引在同 revision 可见	任一路 stage 失败、发布中断、旧读在途
[D]	多 probe 批查、RRF、轮询去重、CandidateRef 稳定	跨 thread 命中、跨 revision 混读、route 超时
[E]	证据闭包、一次 gap 补查、逐主张引用	冲突、缺失桥接、无 source Atom、filter 越界
[F]	刺激原子化、三路并检、relation 过滤、0-2 hints	意图进入 probe、当前 Scene 自召回、主题型误命中

共享验收

- 同一个 CandidateRef schema 同时服务 Deep 与 Shallow。
- 两条召回链在相同 revision 上重复执行得到稳定候选。
- 所有 evidence 与 hint 都能定位到协议定义的 source_ref。

范围外

- Runtime 主链接入与调用编排。
- 图数据库、实体图构建与图遍历召回。
- 历史全量回填、数据迁移与管理接口。
- 安全策略本身的规则设计。